

- `for ctr in range(11, 38):`
- `print(ctr)`

The reason we have 38 there is that the range function generates numbers up to but not including the ending number. If you wanted to print every second number in this range you could use `range(11, 38, 2)`.

If you wanted to iterate in reverse, you can specify the range as `range(37, 10, -1)`, although that is ugly and can cause confusions, so it's recommended you instead use the reversed function, as `reversed(range(11, 38))`.

while Loop

The while loop, like the for loop keeps things simple. The contents of the while loop are repeatedly executed until the condition specified in the loop is no longer true. For example the following will print the numbers 0 to 9.

- `ctr = 0`
- `while ctr < 10:`
- `print(ctr)`
- `ctr = ctr + 1`

Data Structures

There are a couple of important data structures in Python that it is important to know about, and these are lists, tuples, dictionaries, and sets. Let's look at each one of these in turn.

Lists

Python lists are similar to what many languages call arrays. They represent a sequence of values, and can contain a number of types within the same list.

For instance all of the following are valid arrays:

- `arr1 = ["this", "is", "an", "array"]`
- `arr2 = ["so", "is", "this", 1]`
- `arr3 = [1, 2, 3, 4]`

To add a value to a list you can use the append method: `'arr3.append(5)'`

To remove the last value and return it you can use the pop method: `'arr3.pop()'`

Most interesting though are list comprehensions, these allow you to build lists quickly and cleanly. For instance to build a list containing squares of the first 100 numbers you can do the following:

- `squares = [i**2 for i in range(100)]`